Luis Pedro Figueroa

Lab Patterns

Repositorio en github: https://github.com/LuispeFigueroa/labpatterns

Simple Factory

A. ¿Qué sucede si aparece un nuevo síntoma (por ejemplo, mareos)?

Cada que se quisiera tener un nuevo síntoma, se tendría que modificar Covid19Pacient y Medicamento, ya que cada una de estas clases tiene su propia integración del método createSymtom junto a su lista de síntomas ya aceptados. Se tiene que aplicar el cambio a los 2 porque si no la aplicación solo aceptaría el síntoma en una de las 2 clases. Es decir, el nuevo síntoma sería solo aceptado en paciente pero no en medicamento.

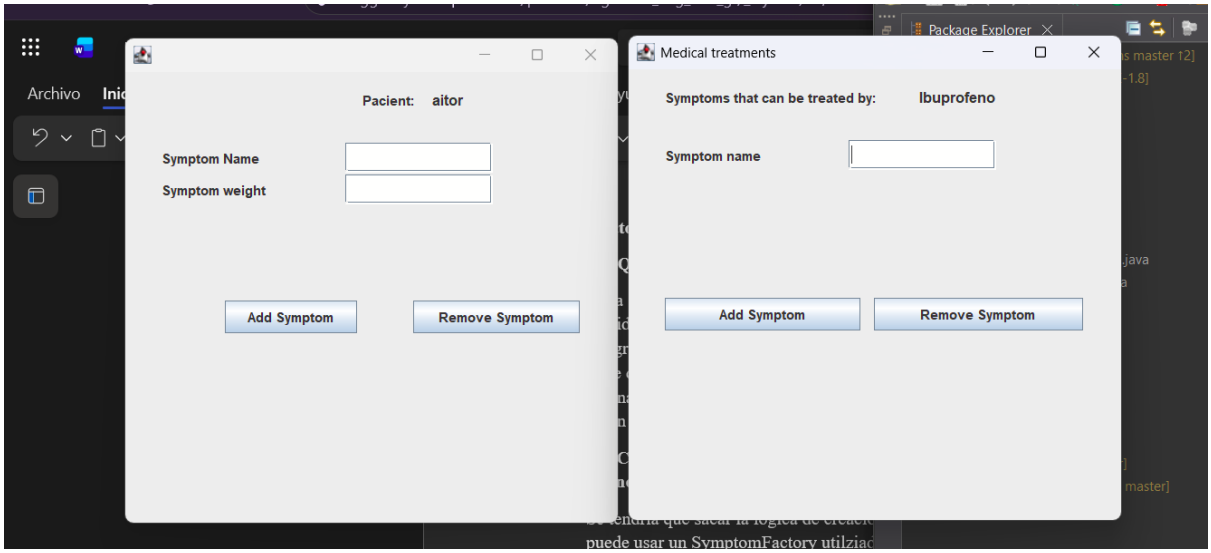
B. ¿Cómo se puede crear un nuevo síntoma sin cambiar las clases existentes (principio OCP)?

Se tendría que sacar la lógica de creación de síntomas de las clases que lo usan. Se puede usar un SymptomFactory utilziado por Covid19Pacient y Medicament para obtener todos los nuevos síntomas que se quieran. De esta forma, cuando se quiera agregar un nuevo síntoma ya no tenemos que modificar las clases, respetando el principio OCP.

C. ¿Cuántas responsabilidades tienen las clases de Covid19Pacient y Medicament (principio SRP)?

Tienen 2 responsabilidades, lo que no cumple con SRP, ya que según el principio cada clase debería tener una sola. Covid19Pacient tiene que manjera los datos del paciente, los síntomas y en base a eso calcular su peso. Aparte, tiene que manejar la creación de síntomas. Medicament es similar, tiene que manejar datos del medicamento y los síntomas que cura. Aparte, tiene que manejar la creación de síntomas para cada medicamento.

Luis Pedro Figueroa



Patrón Observer

Se pide: cambia el programa principal para crear 2 pacientes Covid19Pacient con sus interfaces PacientSymptomGUI y PacientObserverGUI.

```
public class Main {

    /**
     * Launch the application.
     */
    Run | Debug
    public static void main(String[] args) {
        SymptomFactory factory=new SymptomFactory();

        Covid19Pacient pacient=new Covid19Pacient(name: "aitor", years: 35, factory);

        new PacientObserverGUI(pacient);
        new PacientThermometerGUI(pacient);
        new PacientSymptomGUI(pacient);

        Covid19Pacient pacient2=new Covid19Pacient(name: "ane", years: 29, factory);

        new PacientObserverGUI(pacient2);
        new PacientSymptomGUI(pacient2);

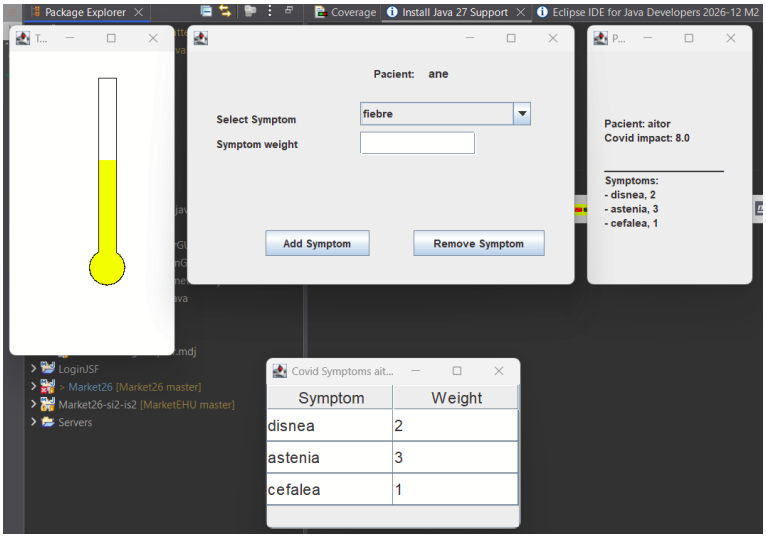
        pacient.addSymptomByName(symptom: "disnea", w: 2);
        pacient.addSymptomByName(symptom: "cefalea", w: 1);
        pacient.addSymptomByName(symptom: "astenia", w: 3);

        ShowPacientTableGUI gui=new ShowPacientTableGUI(pacient);
        gui.setPreferredSize(
                new java.awt.Dimension(width: 300, height: 200));
        gui.pack();
        gui.setVisible(b: true);
    }

}
```

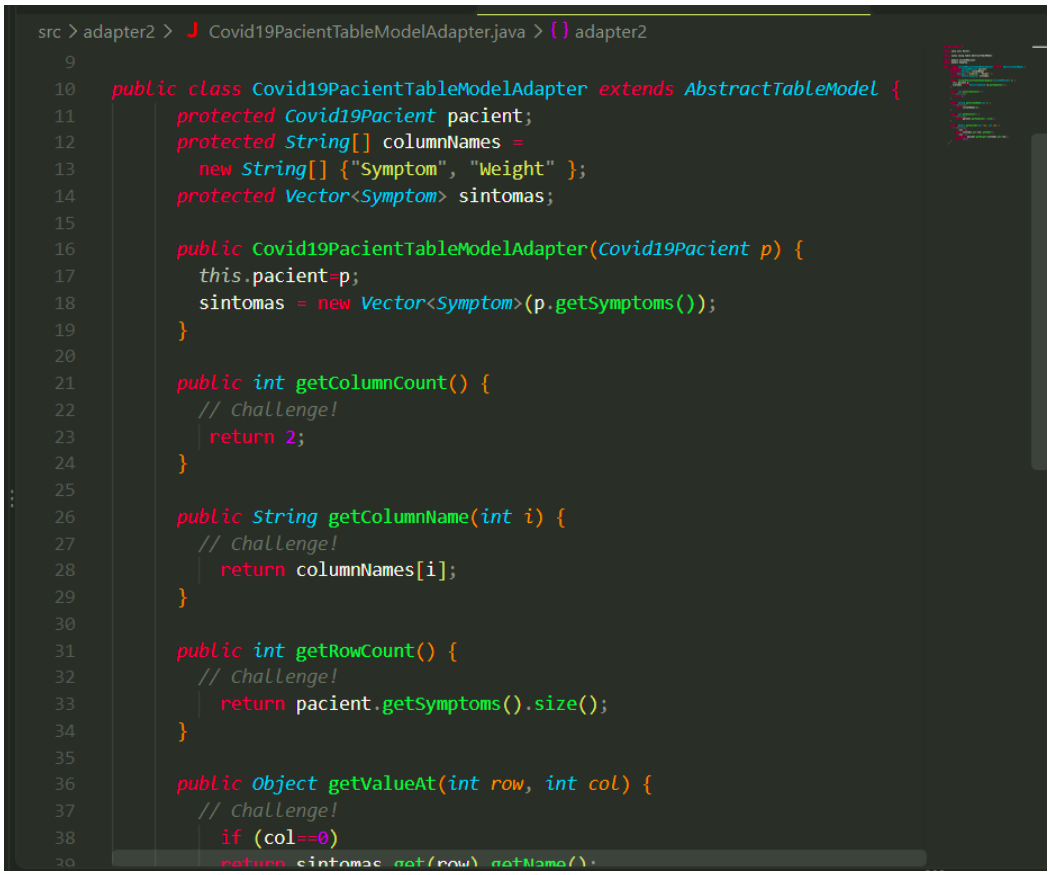
Java: Ready jon (6 years ago) Ln 38, Col 6 Tab Size: 4 UTF-8 LF { } Java

Luis Pedro Figueroa



Patrón Adapter

Añade el código necesario en la clase Covid19PacientTableModelAdapter y ejecuta la aplicación para comprobar que funciona correctamente



Añade otro paciente con otro síntomas, y ejecuta la aplicación para que aparezcan los 2 pacientes son sus síntomas.

```java
public class Main {

    Run | Debug
    public static void main(String[] args) {
        Covid19Pacient pacient=new Covid19Pacient(name: "aitor", years: 35);

        pacient.addSymptomByName(symptom: "disnea", w: 2);
        pacient.addSymptomByName(symptom: "cefalea", w: 1);
        pacient.addSymptomByName(symptom: "astenia", w: 3);

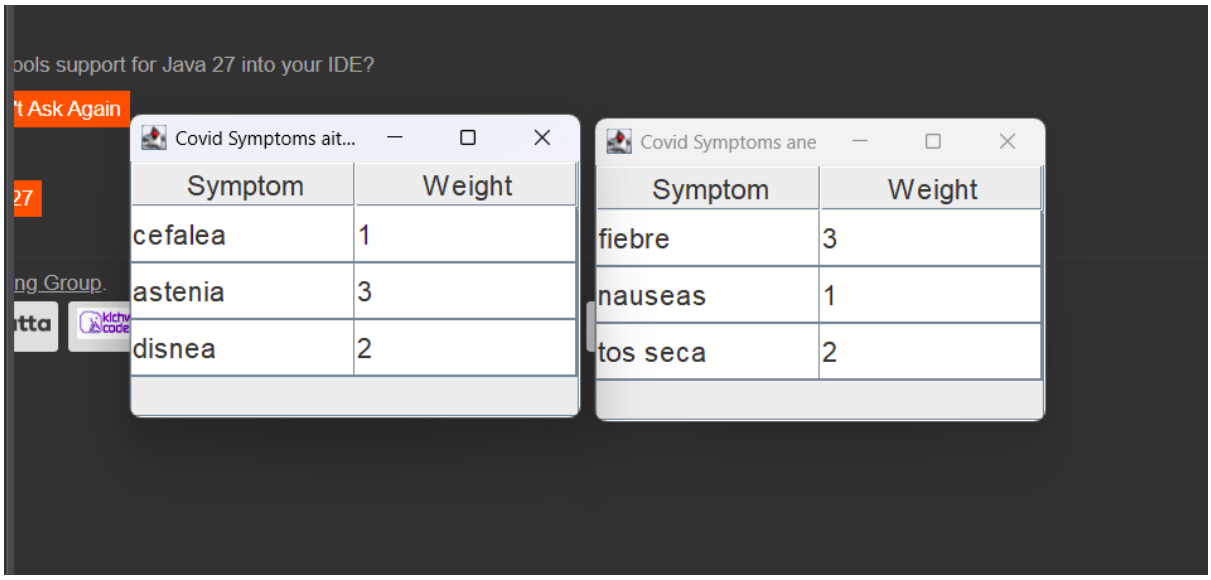
        ShowPacientTableGUI gui=new ShowPacientTableGUI(pacient);
        gui.setPreferredSize(
                new java.awt.Dimension(width: 300, height: 200));
        gui.pack();
        gui.setVisible(b: true);

        Covid19Pacient pacient2=new Covid19Pacient(name: "ane", years: 29);

        pacient2.addSymptomByName(symptom: "fiebre", w: 3);
        pacient2.addSymptomByName(symptom: "tos seca", w: 2);
        pacient2.addSymptomByName(symptom: "nauseas", w: 1);

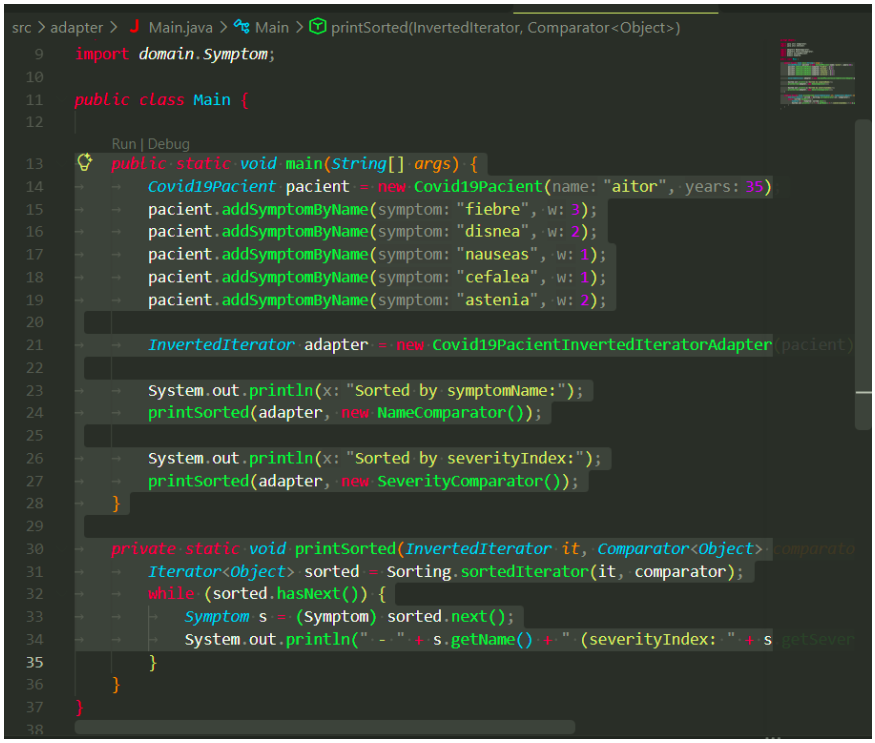
        ShowPacientTableGUI gui2=new ShowPacientTableGUI(pacient2);
        gui2.setPreferredSize(
                new java.awt.Dimension(width: 300, height: 200));
        gui2.pack();
        gui2.setVisible(b: true);

    }
```

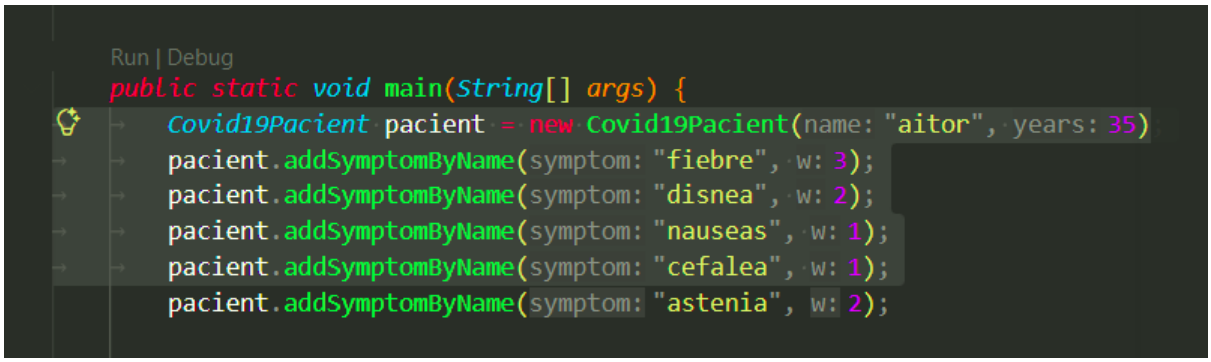


Patrón Iterator y Adapter

Crea un programa principal utilizando el método Sorting.sortedIterator que imprima los 5 síntomas que debe tener un paciente Covid19Pacient. Se imprimirá primero ordenando por symptomName y luego por severityIndex.



Crea un paciente Covid19Pacient con cinco síntomas. La clase Covid19Pacient NOPUEDE CAMBIARSE NADA.



Implementa las interfaces Comparator: una para la ordenación por symptomName, y otrapara la ordenación según severityIndex.

src > adapter2 > NameComparator.java > NameComparator

```java
package adapter2;

import java.util.Comparator;

import domain.Symptom;

public class NameComparator implements Comparator<Object> {


    @Override
    public int compare(Object arg0, Object arg1) {

        String s1 = ((Symptom)arg0).getName();
        String s2 = ((Symptom)arg1).getName();

        return s1.compareTo(s2);

    }
}

```

Crea el patrón adapter sobre la clase Covid19Pacient, implementando la interfaz InvertedIterator. Recuerda crear una constructora adecuada para enviarlela información del paciente

src > adapter2 > SeverityComparator.java > SeverityComparator

```java
package adapter2;

import java.util.Comparator;

import domain.Symptom;

public class SeverityComparator implements Comparator<Object> {

    public int compare(Object arg0, Object arg1) {
        int s1 = ((Symptom)arg0).getSeverityIndex();
        int s2 = ((Symptom)arg1).getSeverityIndex();

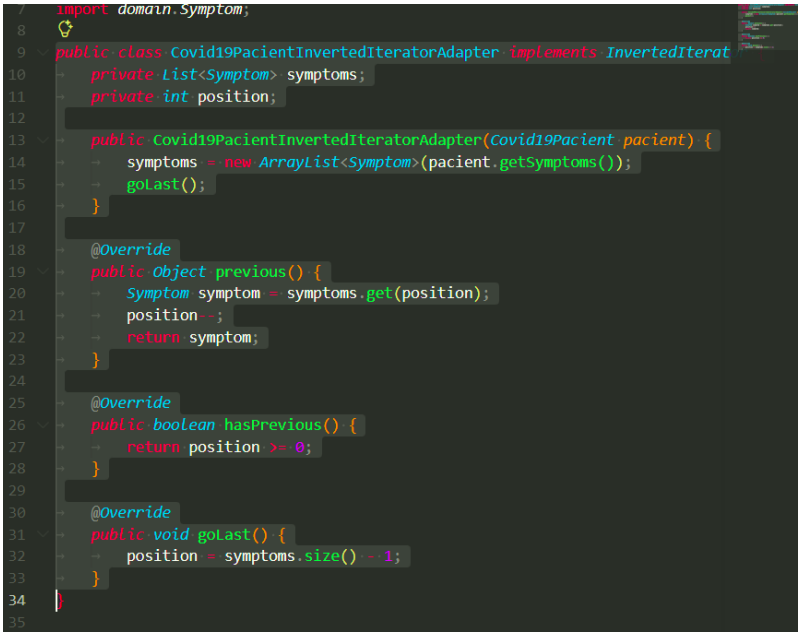
        if(s1>s2) return 1;
        if(s1<s2) return -1;
        return 0;
    }


}
```

Luis Pedro Figueroa

```
Sorted by symptomName:
 - astenia (severityIndex: 5)
 - cefalea (severityIndex: 3)
 - disnea (severityIndex: 3)
 - fiebre (severityIndex: 5)
 - nauseas (severityIndex: 1)
Sorted by severityIndex:
 - nauseas (severityIndex: 1)
 - cefalea (severityIndex: 3)
 - disnea (severityIndex: 3)
 - fiebre (severityIndex: 5)
 - astenia (severityIndex: 5)
```

Adapter implementando InvertedIterator:



```
import domain.Symptom;

public class Covid19PacientInvertedIteratorAdapter implements InvertedIterat
    private List<Symptom> symptoms;
    private int position;

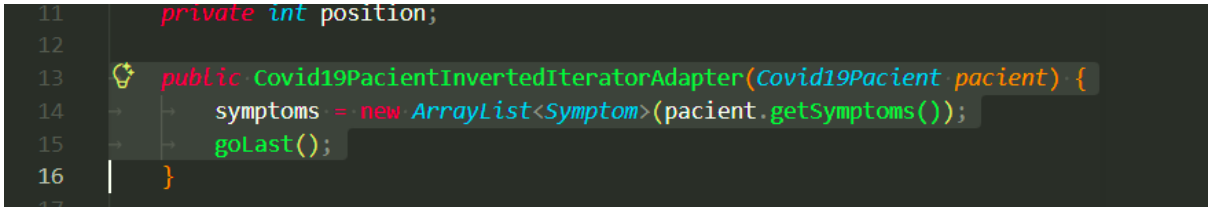
    public Covid19PacientInvertedIteratorAdapter(Covid19Pacient pacient) {
        symptoms = new ArrayList<Symptom>(pacient.getSymptoms());
        goLast();
    }

    @Override
    public Object previous() {
        Symptom symptom = symptoms.get(position);
        position--;
        return symptom;
    }

    @Override
    public boolean hasPrevious() {
        return position >= 0;
    }

    @Override
    public void goLast() {
        position = symptoms.size()-1;
    }
}
```

Constructora que recibe el paciente:



```
    private int position;

    public Covid19PacientInvertedIteratorAdapter(Covid19Pacient pacient) {
        symptoms = new ArrayList<Symptom>(pacient.getSymptoms());
        goLast();
    }
```